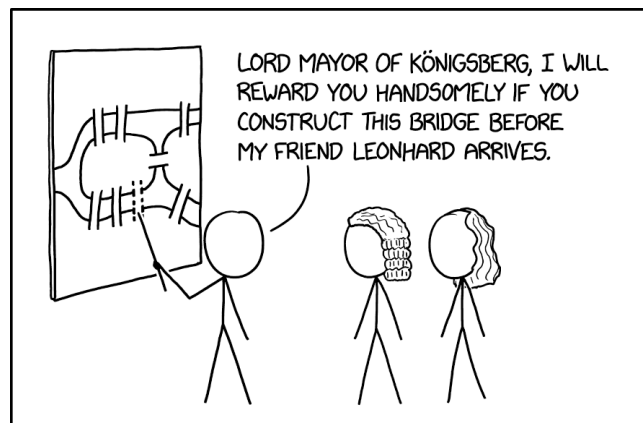


Chapitre 4

Compléments d'algo des graphes



I TRIED TO USE A TIME MACHINE TO CHEAT ON MY ALGORITHMS FINAL BY PREVENTING GRAPH THEORY FROM BEING INVENTED.

1 Couplage d'un graphe biparti

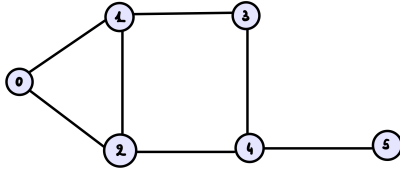
1.1 Recherche de couplage maximum

Définition - Couplage

Soit $G = (S, A)$ un graphe non-orienté.

Un couplage de G est un ensemble $C \subset A$ tel que $\forall a, b \in C^2, a \cap b = \emptyset$

Exemple 1 :



- > $\{\{0, 1\}\}$ est un couplage.
- > $\{\{1, 3\}, \{2, 4\}\}$ est un couplage.
- > $\{\{0, 2\}, \{1, 3\}, \{4, 5\}\}$ est un couplage.
- > etc...

Définition

Soient $G = (S, A)$ un graphe non-orienté et C un couplage de G :

- C est dit **maximal** si $\forall a \in A \setminus C, C \cup \{a\}$ n'est plus un couplage
- C est dit **maximum** si c'est un couplage de cardinal maximum parmi tous les couplages possibles
- C est dit **parfait** si tous les sommets du graphe sont dans le couplage
- Soit $s \in S$, s est dit **apparié** s'il apparaît dans une arête de C . Il est dit **libre** sinon.

Théorème

Un couplage maximum est toujours maximal

DÉMONSTRATION.

Soit C un couplage maximum dans $G = (S, A)$ un graphe non-orienté.

Supposons par l'absurde que C ne soit pas maximal. Alors, il existe $a \in A \setminus C$ tel que $C' = C \cup \{a\}$ soit un couplage.

Or $\text{Card}(C') = \text{Card}(C) + 1$ car $a \notin C$. En particulier, $\text{Card}(C') > \text{Card}(C)$ ce qui est absurde car C est maximum.

donc C est bien maximal.

Définition - Chemin alternant

Soient $G = (S, A)$ un graphe non-orienté et C un couplage de G Un chemin **simple** de G est dit **alternant** pour C s'il emprunte alternativement des arêtes de C et des arêtes de $A \setminus C$

Exemple 2 : Dans l'exemple 1, pour $C = \{\{1, 2\}, \{4, 5\}\}$ les chemins $(0, 1, 2, 4, 5)$ et $(1, 2, 4)$ sont alternants.

Définition - Chemin augmentant

Soient $G = (S, A)$ un graphe non-orienté et C un couplage de G Un chemin **simple** de G est dit **augmentant** pour C s'il est alternant et que ses extrémités sont des sommets libres **différents**

Exemple 3 : $(3, 1, 2, 0)$ est un chemin augmentant dans l'exemple 1.

Théorème

C est maximum dans $G = (S, A) \Leftrightarrow$ il n'existe pas de chemin augmentant

DÉMONSTRATION.

$\Rightarrow$ Par contraposée, supposons qu'il existe un chemin augmentant pour C , notons A_{ch} l'ensemble de ses arêtes. On construit C' :

$$\begin{aligned} C' &= C \Delta A_{ch} \\ &= (C \cup A_{ch}) \setminus (C \cap A_{ch}) \\ &= (C \cap (A \setminus A_{ch})) \cup (A_{ch} \cap (A \setminus C)) \end{aligned}$$

et ceci est une union disjointe!

On remarque, du fait que A_{ch} est augmentant (faire des schémas pour se convaincre), que

$$\text{Card}(C \cap A_{ch}) + 1 = \text{Card}(A_{ch} \cap (A \setminus C))$$

Ainsi,

$$\begin{aligned} \text{Card}(C') &= \text{Card}(C \cap (A \setminus A_{ch})) + \text{Card}(A_{ch} \cap (A \setminus C)) \\ &= \text{Card}(C \cap (A \setminus A_{ch})) + \text{Card}(C \cap A_{ch}) + 1 \\ &= \text{Card}(C) + 1 \end{aligned}$$

D'où, si C' est un couplage de G , la non-maximalité de C .

Il reste à justifier que C' est bien un couplage de G . Soient $a = (x, y), b = (u, v) \in C'$ tels que $a \neq b$.

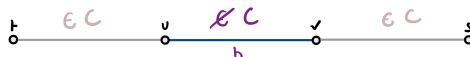
— 1er cas : si $a, b \in C \cap C'$

Alors $a \cap b = \emptyset$ car C est un couplage par hyp.

— 2ème cas : si $a \in C \cap C'$ et $b \notin C$.

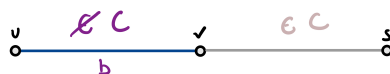
Ainsi, $b \in A_{ch} \cap (A \setminus C)$:

➤ Si b est au milieu du chemin :



u et v étaient appariés dans C (cf. dessin) et donc sont bien diff. de x et y . D'où l'intersection vide.

➤ Si u est l'extrémité gauche du chemin mais v n'est pas l'extrémité droite :



u est libre dans C donc $u \notin a$, v est apparié dans $A \cap A_{ch}$ donc $v \notin a$. D'où l'intersection vide.

➤ Si v est l'extrémité droite du chemin mais u n'est pas l'extrémité gauche : v est libre ... de même.

➤ Si u est l'extrémité gauche du chemin et v l'extrémité droite : tous les deux sont libres dans C et donc n'appartiennent pas à a .

— 3ème cas : si $a \notin C$ et $b \in C \cap C'$.

Alors, puisqu'un chemin augmentant est élémentaire (démonstration bonus) on a le résultat voulu.

En conclusion :

Trouver un couplage maximum

$C \leftarrow \emptyset$

TANT QUE il existe un chemin augmentant dont les arêtes sont notées A_{ch} :

$C \leftarrow C \Delta A_{ch}$

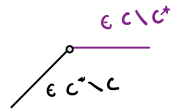
return C

⇐ Par contraposée, supposons que C ne soit pas maximum dans G .

Alors il existe C^* un couplage tq. $\text{Card}(C^*) > \text{Card}(C)$.

On pose $D = C \Delta C^*$ et soit $\tilde{G} = (S, D)$.

Alors, puisque $\forall s \in S, \deg_{\tilde{G}}(s) \leq 2$, chaque composante connexe de $\tilde{G}$ est réduite à un chemin alternant pour C .



Or $|D \cap C^*| > |D \cap C|$ (par hypothèse sur C^*).

Ainsi, il existe dans $\tilde{G}$ une composante connexe avec strictement plus d'arêtes dans C^* que dans C et une telle composante connexe est un chemin augmentant. Donc G admet un chemin augmentant.

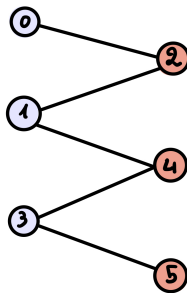
1.2 Recherche de chemin augmentant dans un graphe biparti

Définition - Graphe biparti

Un graphe $G = (S, A)$ est dit **biparti** si $\exists S_1, S_2 \subset S$ tel que :

- $(S = S_1 \cup S_2) \wedge (S_1 \cap S_2 = \emptyset)$
- $\forall (s, s') \in (S_1)^2, \{s, s'\} \notin A$
- $\forall (s, s') \in (S_2)^2, \{s, s'\} \notin A$

Exemple 4 :

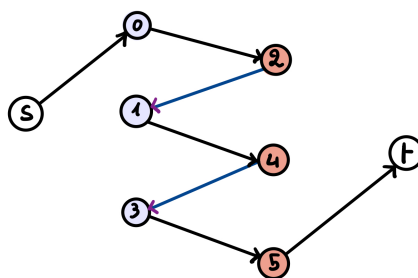


Définition - Graphe orienté associé

Soient $G = (S, A)$ et C un couplage de G , on leur associe un graphe orienté G_c tel que :

- $S(G_c) = S \cup \{s, t\}$ avec $s, t \notin S$
- $\forall x \in S_1$ libre pour C , on met une arête $s \rightarrow x$
- $\forall x \in S_2$ libre pour C , on met une arête $x \rightarrow t$
- Pour toute arête $\{x, y\} \in C, (x \in S_1, y \in S_2)$ on l'oriente de S_2 vers S_1 , i.e $y \rightarrow x$
- Pour toute arête $\{x, y\} \notin C, (x \in S_1, y \in S_2)$ on l'oriente de S_1 vers S_2 , i.e $x \rightarrow y$

Exemple 5 : En reprenant le graphe de l'exemple 4, on ajoute le couplage correspondant aux arêtes violettes et on obtient ceci



Proposition

Soient $G = (S, A)$ non-orienté, C un couplage et G_c le graphe orienté associé.

1. G admet un chemin augmentant pour $C \Leftrightarrow$ il existe un chemin de s à t dans G_c
2. Si $(s, s_0, s_1, \dots, s_k, t)$ est un chemin dans G_c **ALORS** $(s_0, \dots, s_k)$ est un chemin augmentant pour C dans G .

DÉMONSTRATION.

$\Leftarrow$ Soit $s \rightarrow s_1 \rightarrow \dots \rightarrow s_n \rightarrow t$ dans G_c . Montrons que $c = s_1 \rightarrow \dots \rightarrow s_n$ est un chemin augmentant.

Par définition de G_c , s_1 et s_n sont libres. $s_1 \in E_2$ et $\forall i, s_{2i+1} \xrightarrow{\in E_1} s_{2i+2} \xrightarrow{\in E_2}$ n'est pas dans C et $s_{2i} \rightarrow s_{2i+1}$ est dans C .

c est donc bien un chemin augmentant pour C .

$\Rightarrow$ Soit $s_1 \rightarrow \dots \rightarrow s_n$ un chemin augmentant avec $s_1 \in E_1$ et $s_2 \in E_2$ sans perte de généralité.

Comme s_1 et s_n sont libres, alors $(s, s_1) \in A(G_c)$ et $(s_n, t) \in A(G_c)$, et par alternance, $\forall i, \begin{cases} s_{2i+1} \rightarrow s_{2i+2} \notin C \\ s_{2i} \rightarrow s_{2i+1} \in C \end{cases}$ et donc ce sont des arcs de G_c .

Ainsi, $s \rightarrow s_1 \rightarrow \dots \rightarrow s_n \rightarrow t$ est bien un chemin de G_c

Théorème

L'algorithme permettant de trouver un couplage maximum dans un graphe biparti est en $\mathcal{O}(|S||A|)$

DÉMONSTRATION.

La boucle "tant que C n'est pas maximum" principale est de longueur au plus $\frac{|S|}{2}$.

À chaque tour de boucle :

- on construit $G_c : \mathcal{O}(|S| + |A|)$
- on fait un parcours de $G_c : \mathcal{O}(|S| + |A|)$

Au total : $\mathcal{O}(|S|^2 + |S||A|)$, d'où $\mathcal{O}(|S||A|)$.

2 Recherche de composantes fortement connexes dans un graphe orienté : Algorithme de Kosaraju

2.1 Dans un graphe non-orienté

```
1 type graphe = int list array
2 cc: graphe -> int array
```

tel que `cc g c` renvoie `tab` avec `tab.(i)` contenant le nom d'un sommet de la composante connexe de `i` et chacun de ses sommets de cette composante connexe a la même valeur dans `tab`.

```
1 let cc (g : int list array) : int array =
2   let len = Array.length g in
3   let tab = Array.make len (-1) in
4   let rec pp_rec (num : int) (s : int) : unit =
5     if tab.(s) = -1 then
6       begin
7         tab.(s) <- num ;
8         List.iter (pp_rec num) g.(s)
9       end
10  in
11  for i=0 to len-1 do
12    if tab.(i) = -1 then pp_rec i i
13  done;
14  tab
```

2.2 Algorithme de Kosaraju

Définition - Caractérisation d'une composante

- Une composante fortement connexe de G est dit **puits** $\Leftrightarrow$ il n'existe aucune arête sortante de la C.FC
- Une composante fortement connexe de G est dit **source** $\Leftrightarrow$ il n'existe aucune arête entrante dans celle-ci.

Premier jet d'algorithme

TANT QUE G n'est pas vide :

- Choisir un sommet s dans une C.FC puit C de G
- Faire un **pp_rec** depuis s et tous les sommets ainsi atteints forment C
- Retirer de G tous ses sommets obtenus dans C

Théorème

Si on fait un parcours en profondeur de $G = (S, A)$ (dans un ordre quelconque) alors le sommet ayant la plus grande valeur de fin de traitement sera dans une C.FC source.

DÉMONSTRATION.

Soit v un tel sommet et soit C sa C.FC.

Supposons, par l'absurde, qu'il existe $x \notin C, y \in C$ tq $(x, y) \in A$.

On a alors :

1. $d(y) < f(x) < f(y)$
2. v est nécessairement le premier élément de C.FC à être découvert (sinon il deviendrait un descendant du premier élément et donc ne posséderait pas la plus grande date de fin de traitement), ainsi y (comme tous les éléments de C) est un descendant de V :

$$d(v) < d(y) < f(y) < f(v)$$

On obtient alors

$$d(v) < d(y) < f(x) < f(v)$$

et comme $d(x) < d(v) < f(x) < f(v)$ est impossible, on obtient :

$$d(v) < d(x) < f(x) < f(v)$$

Ainsi, x est descendant de v dans le parcours donc $v \rightsquigarrow_G x$. Or $x \rightarrow y \rightsquigarrow_G v$ car y et v sont dans la même composante connexe.

On en déduit que $x \in C$, absurde.

On obtient l'algo suivant :

Algorithme de Kosaraju (pseudo-code)

- **Entrée :** G orienté
- **Algo :**
 - On calcule $\bar{G}$
 - On fait un parcours en profondeur de $\bar{G}$ et on récupère les sommets dans une liste par ordre de fin de traitement obtenues à l'étape précédente pour sélectionner un nouveau sommet pour lancer **pp_rec**

Définition

Soit $G = (S, A)$ un graphe orienté alors on définit $\tilde{G}$ le graphe de ses C.FC comme tel :

- chaque sommet de $\tilde{G}$ correspond à une C.FC de G
- Pour C et C' 2 C.FC on met une arête de C vers C' dans $\tilde{G}$ ssi il existe une arête de $x \in C$ vers $y \in C'$ dans G

Par définition, ce graphe $\tilde{G}$ est un DAG (cyclique), on remarque alors que notre algorithme consiste à faire un *tri topologique* (inverse) de $\tilde{G}$.

2.3 Application : logique propositionnelle

Définition 2-SAT

Une formule φ du calcul propositionnel est sous forme **2-SAT** ssi elle s'écrit comme la conjonction de clauses de taille au plus 2.

Définition - Graphe d'implications

Soit φ une formule sous forme 2-SAT, on définit le graphe des implications de φ , noté G_φ de la manière suivante

- Chaque littéral de φ et sa négation sont représentés par deux sommets
- pour chaque clause $l_1 \vee l_2$, on met une arête de $\neg l_1$ vers l_2 et une autre de $\neg l_2$ vers l_1

On rappelle qu'en logique propositionnelle : $a \vee b \equiv \neg a \rightarrow b \equiv \neg b \rightarrow a$

Proposition

S'il existe un chemin de x à y dans G_φ alors $\varphi \models x \rightarrow y$, i.e tout modèle de φ est modèle de $x \rightarrow y$

DÉMONSTRATION.

Soit $x = x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_p = y$ un chemin de x à y dans G_φ .

Alors, $\forall i \in [0; p-1]$, φ contient $\neg x_i \vee x_{i+1} \equiv x_i \rightarrow x_{i+1}$.

Ainsi, tout modèle de φ est modèle de $x_i \rightarrow x_{i+1}$ donc on aura (par transitivité de $\rightarrow$) $\varphi \models x_0 \rightarrow x_p$, i.e $\varphi \models x \rightarrow y$.

Théorème

φ est satisfiable $\Leftrightarrow G_\varphi$ ne possède aucune C.F.C comportant à la fois une variable et sa négation.

DÉMONSTRATION.

$\Rightarrow$ Par contraposée, supposons qu'il existe une C.F.C de G_φ qui contient une variable x et $\neg x$.

Alors il existe un chemin de x à $\neg x$ dans G_φ . Donc, d'après la proposition précédente, tout modèle de φ est modèle de $x \rightarrow \neg x \equiv \neg x$.

De même, il existe un chemin de $\neg x$ à x dans G_φ donc tout modèle de φ est modèle de $\neg x \rightarrow x \equiv x$.
Ainsi, tout modèle de φ est modèle de x et de $\neg x$, i.e $\varphi \models x \wedge \neg x$.

Donc φ est une contradiction, d'où le fait qu'elle ne soit pas satisfiable.

$\Leftarrow$ Supposons qu'aucune C.F.C ne contienne une variable et sa négation. Construisons $\mu : \nu \rightarrow \mathbb{B}$ un modèle pour φ .

On procède ainsi : on choisit une C.F.C puit de G_φ , notée C . Pour tout $l \in C$, on pose $\mu(l) = V$ et $\mu(\neg l) = F$.

On supprime ensuite dans G_φ tous les sommets à qui l'on a attribué une image et on recommence tant qu'il reste des sommets.

μ est alors bien définie, reste à justifier que c'est bien un modèle de φ .

Soit $l_1 \vee l_2$ une clause quelconque de φ . Supposons que $\mu(l_1)$ est fixée avant $\mu(l_2)$. Ainsi, au moment de considérer $\mu(l_1)$, on est en train de considérer la C.F.C de l_1 ou de $\neg l_1$.

$\Rightarrow$ si on a considéré la C.F.C de l_1 : alors on a posé $\mu(l_1) = V$, donc $l_1 \vee l_2$ sera évaluée à V

$\Rightarrow$ sinon : c'est qu'on a posé $\mu(l_1) = F$ en considérant la C.F.C de $\neg l_1$ qui est puit.

Dans G_φ , on a $\neg l_1 \rightarrow l_2$ avec l_2 toujours dans le graphe quand on fixe $\mu(l_1)$ et, comme la C.F.C de $\neg l_1$ est puit à ce stade, elle contient l_2 . On en déduit que l'algorithme fixe $\mu(l_2) = V$.

D'où le résultat voulu.

3 A*

3.1 Rappel sur Dijkstra

Soit $G = (S, A, p)$ un graphe orienté et pondéré avec p à valeurs positives ou nulles.
Soit $s \in S$, l'algorithme de Dijkstra a pour objectif de calculer $d(s, x)$ pour tout $x \in S$

Algorithme - Dijkstra (pseudo-code 1)

```
 $\forall x \in S, \delta(x) \leftarrow +\infty$   
 $\delta(s) = 0$   
Initialiser une file de priorité  $F$  avec tous les sommets de  $S$   
while  $F \neq \emptyset$  do  
   $x \leftarrow \text{defile}(F)$   
  for each  $y$  voisin de  $x$  do  
    if  $\delta(y) > \delta(x) + p(x, y)$  then  
       $\delta(y) \leftarrow \delta(x) + p(x, y)$   
return  $\delta$ 
```

→ Ici, la file de priorité est une file de priorité minimale et la fonction de priorité est δ .

Proposition - Invariant de l'algorithme de Dijkstra (version 1)

Chaque sommet x défilé satisfait $\delta(x) = d(s, x)$

Pour l'implémentation de F , on utilise en général un tas-min mais modifier une priorité n'est pas sans conséquence sur l'organisation du tas. On remarque alors que comme les valeurs sont modifiées uniquement à la baisse, une percolation vers le haut permet de faire l'opération de modification de priorité.

Proposition - Complexité de l'algorithme de Dijkstra (version 1)

$\mathcal{O}((|A| + |S|) \log |S|)$

DÉMONSTRATION.

On a :

- initialisation de δ : $\mathcal{O}(|S|)$
- initialisation de F : $\mathcal{O}(|S|)$
- défilement des $|S|$ sommets au max : $\mathcal{O}(|S| \log(|S|))$
- on a $\mathcal{O}(|A| \log(|S|))$ pour les modifs. de priorité.

Au total, on a bien $\mathcal{O}((|A| + |S|) \log(|S|))$.

Remarque : On peut utiliser un **tas de Fibonacci** pour implémenter la file de priorité, ce qui permet une modif de prio en $\mathcal{O}(1)$, ce qui donnerait alors une complexité en $\mathcal{O}(|S| \log(|S|) + A)$. Ce modèle reste néanmoins moins performant en pratique.

Néanmoins on préférera une autre version pour l'implémentation

Algorithme - Dijkstra (pseudo-code version 2)

```
 $\forall x \in S, vu(x) \leftarrow \text{false}$   
 $\forall x \in S, \delta(x) \leftarrow +\infty$   
 $\delta(s) = 0$   
 $F \leftarrow \emptyset$   
 $F \leftarrow s$  (file de prio de priorité  $\delta$ )  
while  $F \neq \emptyset$  do  
   $x \leftarrow \text{defile}(F)$   
  if  $vu(x) = \text{false}$  then  
     $vu(x) \leftarrow \text{true}$   
    for each  $y$  voisin de  $x$  do  
      if  $\delta(y) > \delta(x) + p(x, y)$  then  
         $\delta(y) \leftarrow \delta(x) + p(x, y)$   
        insérer  $y$  dans  $F$  avec priorité  $\delta(y)$   
return  $\delta$ 
```

Dans cette version, la taille de F est en $\mathcal{O}(|A|)$

Proposition - Complexité de l'algorithme de Dijkstra (version 2)

$$\mathcal{O}(|A|\log|S|)$$

DÉMONSTRATION.

Les opérations "défiler" pouvant se faire en $\mathcal{O}(\log|A|)$, on a :

$$\mathcal{O}(|S| + |A|\log|A| + |A|\log|A|)$$

Or $A = \mathcal{O}(|S|^2)$, on a alors $\log|A| = \mathcal{O}(\log|S|)$, d'où la complexité recherchée.

3.2 A*

Objectif : Soient $G = (S, A, p)$ un graphe orienté et pondéré et p à valeurs dans $\mathbb{R}_+$. Soient $s, t \in S$, on cherche le poids minimal d'un chemin de s à t .

On veut orienter les calculs dans la direction de t , pour cela on utilise une heuristique $h : s \rightarrow \mathbb{R}_+$ qui est une estimation de la distance à t . Pour cela, la seule modif consiste à utiliser $\delta(x) + h(x)$ au lieu de $h(x)$ comme valeur de priorité.

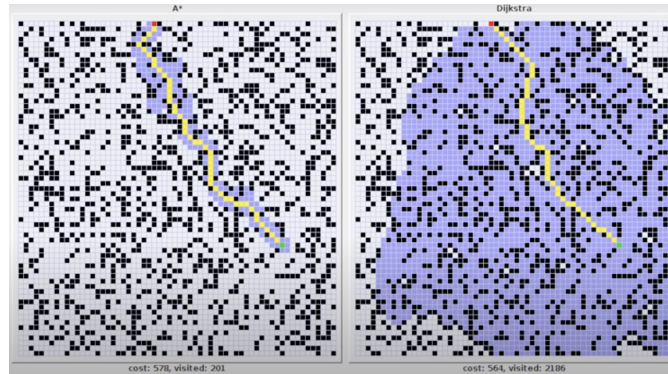


FIGURE 1 – Exemple de l'orientation de calcul pour trouver le plus court chemin.

On obtient l'algorithme suivant :

Algorithme - A* (pseudo-code)

```

 $\forall v \in S, \delta(v) \leftarrow +\infty$ 
 $\delta(s) \leftarrow 0$ 
On met tous les sommets de  $S$  dans  $F$ , une file de priorité min. associé à la priorité  $\delta + h$ 
while  $F \neq \emptyset$  do
   $x = \text{defile}(F)$ 
  if  $x = t$  then
    return  $\delta(x)$ 
  else
    for each  $y$  voisin de  $x$  do
      if  $\delta(y) > \delta(x) + p(x, y)$  then
         $\delta(y) \leftarrow \delta(x) + p(x, y)$ 
        if  $y \in F$  then
          on modifie sa priorité
        else
          on met  $y$  dans  $F$  avec priorité  $\delta + h$ 

```

Différence entre A* et Dijkstra

- val de prio différentes
- **invariant perdu**
- les éléments peuvent être réinsérés après avoir été défilés
- **la terminaison n'est plus garantie**
- **la correction n'est plus garantie**

Définition - Admissibilité

Une heuristique h est dite admissible si $\forall x \in S, h(x) \leq d(x, t)$.

Théorème

Si h est admissible et que t est accessible depuis s dans G alors A* renvoie $d(s, t)$.

DÉMONSTRATION.(cf. Cahier de Prépa)

Soit c un plus court chemin de s à t , notons $c : s = x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_n = t$.

Alors, on montre par récurrence sur $0 \leq i < n$ que chacun des x_i aura sa valeur $\delta(x_i) = d(s, x_i)$ avant que t ne soit défilé :

- Initialisation : immédiat
- Pour $x_1 : x_1$ va être inséré avec une prio $= p(x_0, x_1) + h(x_1) \leq p(x_0, x_1) + d(x_1, t) = d'$
Si t est défilé avec une valeur $d > d'$ alors x_1 est bien défilé avant.

etc...

Néanmoins la complexité peut être exponentielle à cause du fait que les sommets peuvent être enfilés/défilés de F plusieurs fois.

Définition - Heuristique monotone

Une heuristique est monotone si

$$\forall (u, v) \in A, h(u) \leq p(u, v) + h(v)$$

Théorème

h monotone et $h(t) = 0 \implies h$ admissible.

DÉMONSTRATION.

Soit $u \in S$ et soit $c : u = u_0 \rightarrow u_1 \rightarrow \dots \rightarrow u_n = t$ un plus court chemin de u à t .

On veut montrer que $h(u) \leq \sum_{0 \leq i \leq n-1} p(u_i, u_{i+1}) = p(c)$.

On montre alors par rec. sur $i \in \llbracket 0; n \rrbracket$, $h(u) \leq \sum_{0 \leq k \leq i-1} p(u_k, u_{k+1}) + h(u_i)$:

— $i = 0$: $h(u_0) = h(u) \leq h(u_0)$

— Héré : soit $i \in \llbracket 0; n-1 \rrbracket$, alors :

$$\begin{aligned} h(u) &\leq \sum_{0 \leq k \leq i-1} p(u_k, u_{k+1}) + h(u_i) \\ &\leq \sum_{0 \leq k \leq i-1} p(u_k, u_{k+1}) + p(u_i, u_{i+1}) + h(u_{i+1}) \text{ (car } h \text{ est monotone)} \\ &\leq \sum_{0 \leq k \leq i} p(u_k, u_{k+1}) + h(u_{i+1}) \end{aligned}$$

— $i = n$: $h(u) \leq p(c) + h(u_n) = p(c) + h(t) = p(c)$

d'où le résultat voulu.

Théorème

Si h est monotone alors chaque sommet sera défilé une seule fois dans l'exécution de A^* .

DÉMONSTRATION.

Soit v un sommet défilé avec une prio $p = \delta(v) + h(v)$, alors tous les sommets v' rajoutés à F lors de ce tour le sont avec une prio p' tq :

$$p' = \underbrace{\delta(v) + p(v, v') + h(v')}_{\text{nouveau } \delta(v')} \geq \delta(v) + h(v) = p$$

Ainsi, les sommets sont défilés de F avec des valeurs de priorité croissante.

Supposons, par l'absurde, que v soit extrait 2 fois de F : notons p_1 la priorité de la première extraction, p_2 celle de la deuxième.

On a donc $p_2 \geq p_1$, i.e

$$\delta_2(v) + h(v) \geq \delta_1(v) + h(v)$$

D'où

$$\delta_2(v) \geq \delta_1(v)$$

Or, si on a choisi d'introduire une deuxième fois v dans F , c'est qu'on a trouvé $\delta_2(v) < \delta_1(v)$. Absurde.

Proposition - Complexité de A^*

La complexité de A^* avec une heuristique monotone est polynomiale :

$$\mathcal{O}(|S| \log |S| + |A| \log |S|) = \mathcal{O}(|S|^3)$$